温州肯恩大学
WENZHOU-KEAN UNIVERSITY

MGS 3001 WHS01
Python Programming for Business

Structured Data Cleaning

Dawei Chen

Week 11-2, 7 May 2026

Remaining Semester

Python Syntax → Work with AI → Web Data Collection → **Data Analysis**

Date	Topic	Note
5/07	Structured Data Cleaning	Pandas, missing values, duplicates, type conversion
	*Assignment 3 due on **5/10** (Sunday) at 23:59*	
5/12	Text Processing & Variable Construction	Regex, string methods, construct numeric variables from text
5/14	*Research Workshop III*	*Paper anatomy, section-by-section guidance, writing with AI*
5/19	Exploratory Data Analysis	Descriptive statistics, correlations, groupby analysis
5/21	Data Visualization	Matplotlib, Seaborn, visualization principle
5/26	Regression: Theory	Interpretation, model specification
5/28	Regression: Application	statsmodels OLS on project data, results presentation
6/02	*Research Workshop IV*	*Draft feedback, presentation prep, consultation*
6/04 6/09	Final Presentations	8-min pre + 2-min Q&A per project
	*Final report due on **6/10** (Wednesday) at 23:59*	

Learning Objectives

By the end of this session, you should be able to:

1. Load a CSV dataset into a Pandas DataFrame and inspect its structure
2. Identify common data quality issues in a real dataset
3. Explain three types of missing data mechanisms (MCAR, MAR, MNAR)
4. Apply appropriate strategies to handle missing values, duplicates, and type errors

Question: What **issues** or **inconsistencies** do you see in the following dataset?

full_name	stars	language	created_at	description	owner_type
microsoft/vscode	165432	TypeScript	2015-09-03T08:30:00Z	Visual Studio Code	Organization
torvalds/linux	180000	C	2011-09-04	Linux kernel source tree	user
user123/my-notes	N/A		2024-01-15T10:00:00Z		User
microsoft/vscode	165500	TypeScript	2015-09-03T08:30:00Z	Visual Studio Code	Organization
facebook/react	“230k”	JavaScript	2013-05-24T16:15:54Z	A JS library for building UIs	Organization
jdoe/test-repo	-5	python	03/20/2025	testing stuff □	User

Data Cleaning Pipeline

A reference guide, not a comprehensive codebook

A systematic approach — not ad-hoc fixes

Outlier, unrealistic values, etc.

1 **Inspect**
shape, dtypes, head, info

→

2 **Diagnose**
isnull, duplicated, value_counts

→

3 **Missing Values**
drop or fill

→

4 **Duplicates**
detect & remove

→

5 **Type Conversion**
datetime, numeric, category

→

6 **Validate**
re-inspect & document

Today we'll work through each step using a demo dataset.

Project Workflow: One Stage, One Notebook

Organize your project into separate Jupyter notebooks — not one monolithic file.

Step 1
Data Collection

`01_collection.ipynb`

Input: API / Web page

Output: raw_data.csv

- Call API or scrape HTML
- Paginate through results
- Save raw JSON → CSV
- No filtering, no editing

→

Step 2
Data Cleaning

`02_cleaning.ipynb`

Input: raw_data.csv

Output: cleaned_data.csv

- Handle missing values
- Remove duplicates
- Fix data types
- Derive new variables

→

Step 3
Data Analysis

`03_analysis.ipynb`

Input: cleaned_data.csv

Output: figures + tables

- Descriptive statistics
- Visualizations
- Regression
- Interpret results

Why Separate Notebooks?

One Giant Notebook

- ✗ **Fragile** Re-running early cells (API calls) overwrites your raw data or hits rate limits.
- ✗ **Slow** Every time you tweak a chart, you re-run data collection. Wastes time and API quota.
- ✗ **Hard to debug** A bug in cell 47 of a 120-cell notebook is a nightmare to trace.
- ✗ **Unreproducible** Out-of-order cell execution produces hidden state errors that break results silently.
- ✗ **Poor collaboration** Merge conflicts are inevitable when everything lives in one file.

Separate Notebooks

- ✓ **Collect once** Raw data is saved to CSV. You never re-run API calls just to test a new cleaning step.
- ✓ **Fast iteration** Change a chart or regression spec? Re-run only the analysis notebook — takes seconds.
- ✓ **Clear data lineage** raw_data.csv → cleaned_data.csv → figures. Every artifact is traceable.
- ✓ **Easier debugging** Each notebook is short and focused. Errors are isolated to one stage.
- ✓ **Matches your paper** Collection → Methods → Results maps directly to your notebook sequence.

What is Pandas?

Tutorial

Documentation

pandas is Python's main library for working with tabular (structured) data.

- Load CSV, Excel, JSON into a **DataFrame**
- Select, filter, and sort rows and columns
- Handle missing values and duplicates
- Group, aggregate, and transform data
- Export cleaned data back to CSV

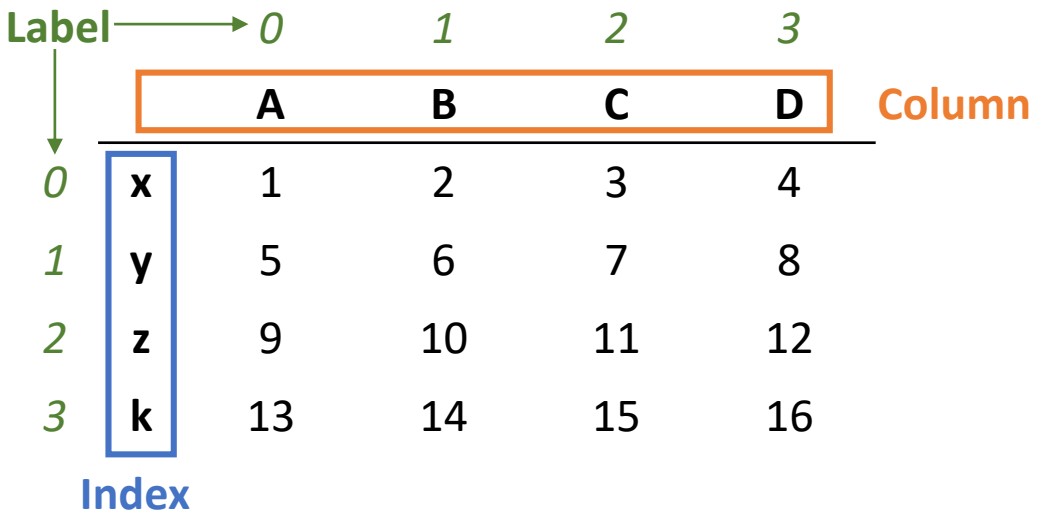
```python
import pandas as pd

# Load your dataset
df = pd.read_csv("repos.csv")

# Quick look
df.shape
df.head()
df.info()
df.describe()
```

Think of a DataFrame as a spreadsheet inside Python — rows and columns, just like Excel.

DataFrame Basic Operations

Tutorial

Documentation



Multiple functions can achieve the same result — master one and use it well.

- Reindexing & Dropping
  - Rename, reorder, add, drop, sort, rank
- Selecting & Filtering
  - Select and extract subsets based on conditions
- Arithmetic & Alignment
  - Perform arithmetic with automatic alignment on labels
- Function Application & Mapping
  - Apply, map, aggregate, and summarize data
- Combination
  - Merge, join, concatenate DataFrames

Step 1. Inspect Your Data

`df.shape`

Returns (rows, columns) — how big is your dataset?

→ (557, 14)

`df.head()`

Shows the first 5 rows — what does the data look like?

→ First 5 rows as table

`df.info()`

Column names, data types, non-null counts — the diagnostic report

→ 14 columns, dtypes, memory

`df.describe()`

Summary stats for numeric columns — min, max, mean, std

→ count, mean, std, min, max

Always inspect before you clean. Know your data's shape, types, and quirks first.

Step 2. Diagnose Data Quality Issues

Missing Values

```
df.isnull().sum()
df.isnull().mean()
```

Key question:

How many? Which columns?
Random or systematic?

Duplicates

```
df.duplicated().sum()
df[df.duplicated()]
```

Key question:

Exact duplicates? Partial?
Based on which key?

Type Mismatches

```
df.dtypes
df['col'].unique()
```

Key question:

Dates stored as strings?
Numbers as objects?

Step 3. Missing Values

Three Types of Missing Mechanisms (Rubin 1976)

MCAR

Missing Completely at Random

Missingness is unrelated to any variable, neither observed nor unobserved.

Some responses were lost due to a system glitch – any participant was equally likely to be affected.

Safe to drop rows (listwise deletion) without introducing bias. Rare in practice.

MAR

Missing at Random

Missingness is related to other observed variables, but not to the missing value itself.

Older respondents were less likely to report income, but within each age group, missingness is random.

Most common in practice. Imputation methods (e.g., fill with mode) are appropriate.

MNAR

Missing Not at Random

Missingness is related to the value that is missing itself.

High-income respondents choose not to disclose their income.

Hardest to handle. Consider adding a binary indicator variable (is_missing = 1) to your regression.

You are conducting a salary survey. Some respondents did not report their income, resulting in missing values in the income column.

Step 3. Missing Values

Delete

```
df.dropna()
```

When to use:

Few missing rows, data is MCAR

Risk:

Lose observations → less statistical power

```
df.drop(columns=['col'])
```

When to use:

Column mostly empty (>50% missing)

Risk:

Lose a variable entirely

Imputation

```
df['col'].fillna(value)
```

When to use:

Meaningful default exists (0, median, mode)

Risk:

Introduces artificial values — must justify

Tips: Always report how many observations were dropped and why.
Avoid overcleaning — do not remove useful variation or rare patterns.

Step 3. Missing Values

Common `fillna()` Patterns

```
# Fill with a constant
df['license_name'].fillna('No license', inplace=True)

# Fill with column median (numeric)
df['stars'].fillna(df['stars'].median(), inplace=True)

# Fill with column mode (categorical)
df['language'].fillna(df['language'].mode()[0], inplace=True)

# Drop rows where a critical column is missing
df.dropna(subset=['full_name', 'created_at'], inplace=True)

# MNAR tip: add a binary flag before imputing
df['license_missing'] = df['license_name'].isnull().astype(int)
df['license_name'].fillna('Unknown', inplace=True)
```

Step 4. Handling Duplicates

Exact duplicates

Every column value is identical.
Usually from double-collection
or overlapping API pages.

Key-based duplicates

Same entity (e.g., same full_name)
but different values in other
columns (e.g., updated star count).

```
# Count exact duplicates
df.duplicated().sum()

# View the duplicates
df[df.duplicated(keep=False)]

# Remove exact duplicates
df = df.drop_duplicates()

# Key-based: keep first by name
df = df.drop_duplicates(subset=['full_name'],
       keep='first')
```

Step 5. Type Conversion

When Pandas loads a CSV, it guesses column types. It often gets them wrong.

Problem	Example	Solution
Date stored as string	“2025-01-15T08:30:00Z”	`pd.to_datetime(df['col'])`
Number stored as string	“1234” (object type)	`pd.to_numeric(df['col'])`
Category as string	“User” / “Organization”	`df['col'].astype('category')`
Boolean as string	“True” / “False”	`df['col'].astype(bool)`
Inconsistent capitalization	“user” vs “User”	`df['col'].str.lower()`

```
# Convert created_at from string to datetime
df['created_at'] = pd.to_datetime(df['created_at'])

# Now you can do date math
df['repo_age_days'] = (pd.Timestamp.now() - df['created_at']).dt.days
```

Step 6. Validate & Document

Re-inspect after cleaning

```
# Final checks
print(df.shape)
print(df.dtypes)
print(df.isnull().sum())
print(df.duplicated().sum())

# Save cleaned dataset
df.to_csv('repos_cleaned.csv',
          index=False)
```

Document in your paper

- How many rows before vs. after cleaning
- Which columns had missing values and how you handled them
- How many duplicates were removed and by which key
- Which type conversions you applied and why
- Any rows dropped and the justification

Reproducibility = your Methods section describes every step so that someone else could replicate your data cleaning from the raw data to the analysis-ready dataset.

Quick Reference: Key Pandas Methods

Task	Method	What It Does
Inspect	`df.info()`	Column types, non-null counts, memory
Inspect	`df.describe()`	Summary statistics for numeric cols
Missing	`df.isnull().sum()`	Count NaN per column
Missing	`df.fillna(value)`	Replace NaN with a value
Missing	`df.dropna()`	Remove rows with NaN
Duplicates	`df.duplicated().sum()`	Count duplicate rows
Duplicates	`df.drop_duplicates()`	Remove duplicate rows
Types	`pd.to_datetime()`	Convert string to datetime
Types	`pd.to_numeric()`	Convert string to number
Export	`df.to_csv('file.csv')`	Save cleaned data to file

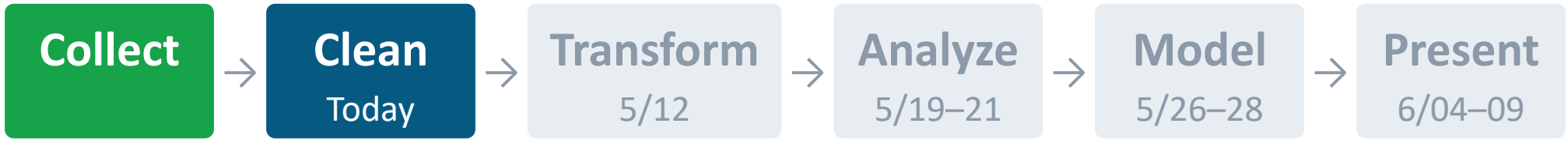
Lab Exercise

Apply today's methods to your project dataset (or the demo dataset github_repos_raw.txt on Canvas)

1	**Load raw data into a DataFrame**	Use pd.read_csv() — check df.shape and df.head()
2	**Run the diagnostic trio**	df.info(), df.isnull().sum(), df.duplicated().sum()
3	**Handle missing values**	Decide: drop or fill? Document your decision.
4	**Remove duplicates if any**	Identify the right key column (e.g., your unique identifier)
5	**Fix data types**	Convert date strings → datetime, fix numeric columns
6	**Save and validate**	Export to repos_cleaned.csv, re-run df.info()

Use AI to help write cleaning code — but you must understand and verify every step (4D: Discernment).

What's Next



- **Assignment 3 due Sunday 5/10 at 23:59**
  Data collection report & GitHub repository
- **Next Tuesday (5/12): Text Processing & Variable Construction**
  Regex, string cleaning, constructing research variables from raw text
- **Next Thursday (5/14): Academic Writing**
  Paper anatomy, section-by-section guidance — bring a rough outline of your paper

Bonus Time

Share & interpret your data cleaning notebook

Show Us

- Open your 02_cleaning.ipynb
- Walk through it cell by cell
- Show the before & after df.info()

Explain

- Why did you choose to drop vs. fill?
- What does each line of code do?
- What changed in the output?

We'll Ask

- What would happen if you removed this line?
- Why this fill value and not another?
- Could this step introduce bias?

Q&A

dchen@kean.edu CBPM B214

Office hours: Tue & Thu 11:15–12:30 | Wed 14:00–16:30